LAPORAN

“Studi Kasus Bioskop 212”



Nama : Kristian Utama Putra
NRP : 3125600047
Dosen : Grezio Arifiyan Primajaya , S.Kom, M.Kom

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TAHUN 2026

TUGAS

1. Rancang dan implementasikan menggunakan pendekatan OOP berdasarkan

kebutuhan bisnis

2. Berdasarkan kelas yang sudah ditentukan, silahkan tentukan : - atribut; - tipe data; - access modifier; - constructor; - method; - parameter method; - return value; - hubungan antar kelas.

3. Buat UML Class Diagram berdasarkan desain yang kalian buat.

4. Implementasikan seluruh desain tersebut ke dalam program Java.

5. Setiap class harus memiliki interaksi dengan kelas yang lain, sesuai dengan

kebutuhan bisnis.

6. Program yang dikembangkan harus memiliki minimal 1 alur proses bisnis utama

sesuai dengan studi kasus

7. Wajib menerapkan : - Enkapsulasi - Polymorphism - array/array of object untuk menyimpan dan mengolah beberapa object.

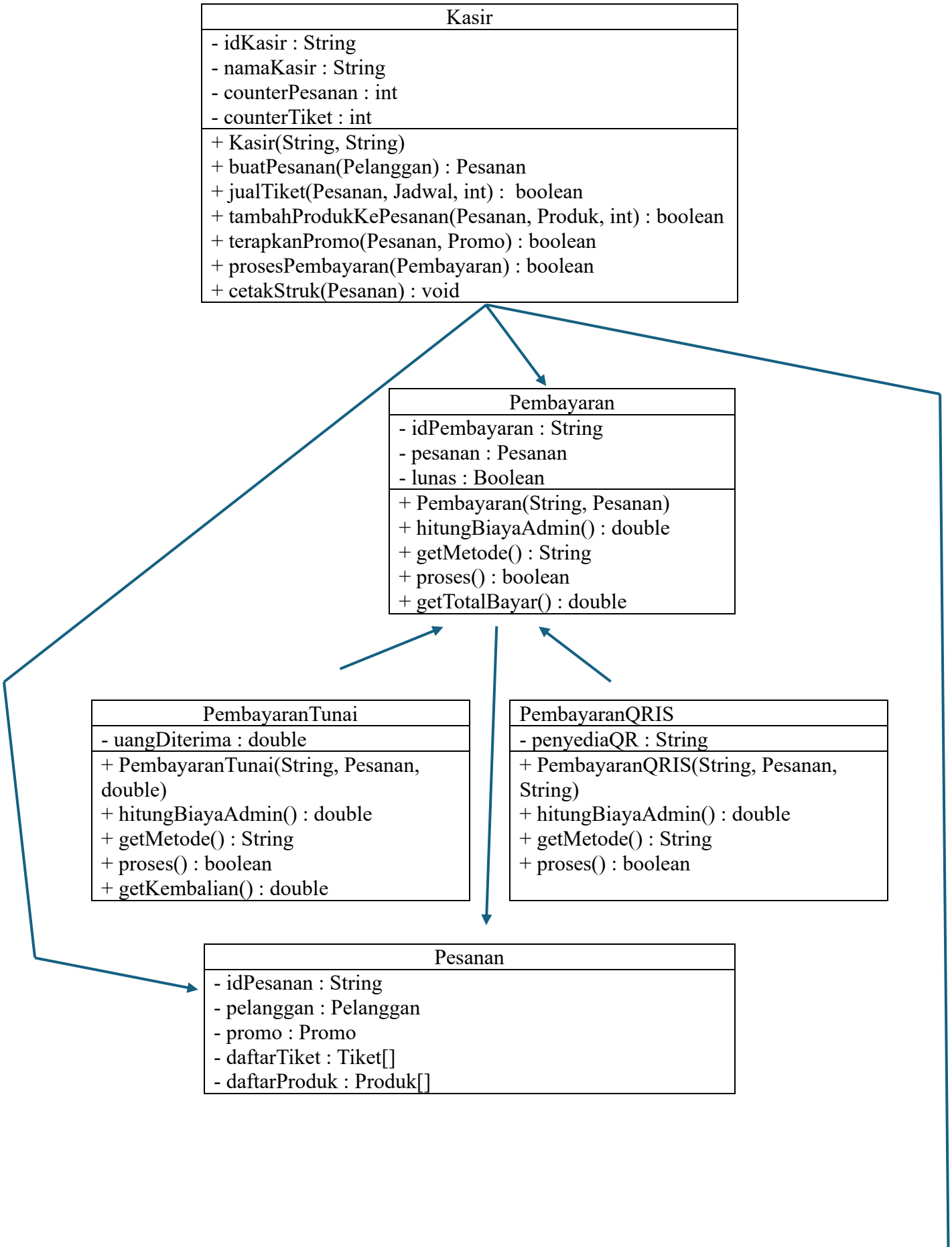
8. Pastikan program yang dikembangkan memiliki main()

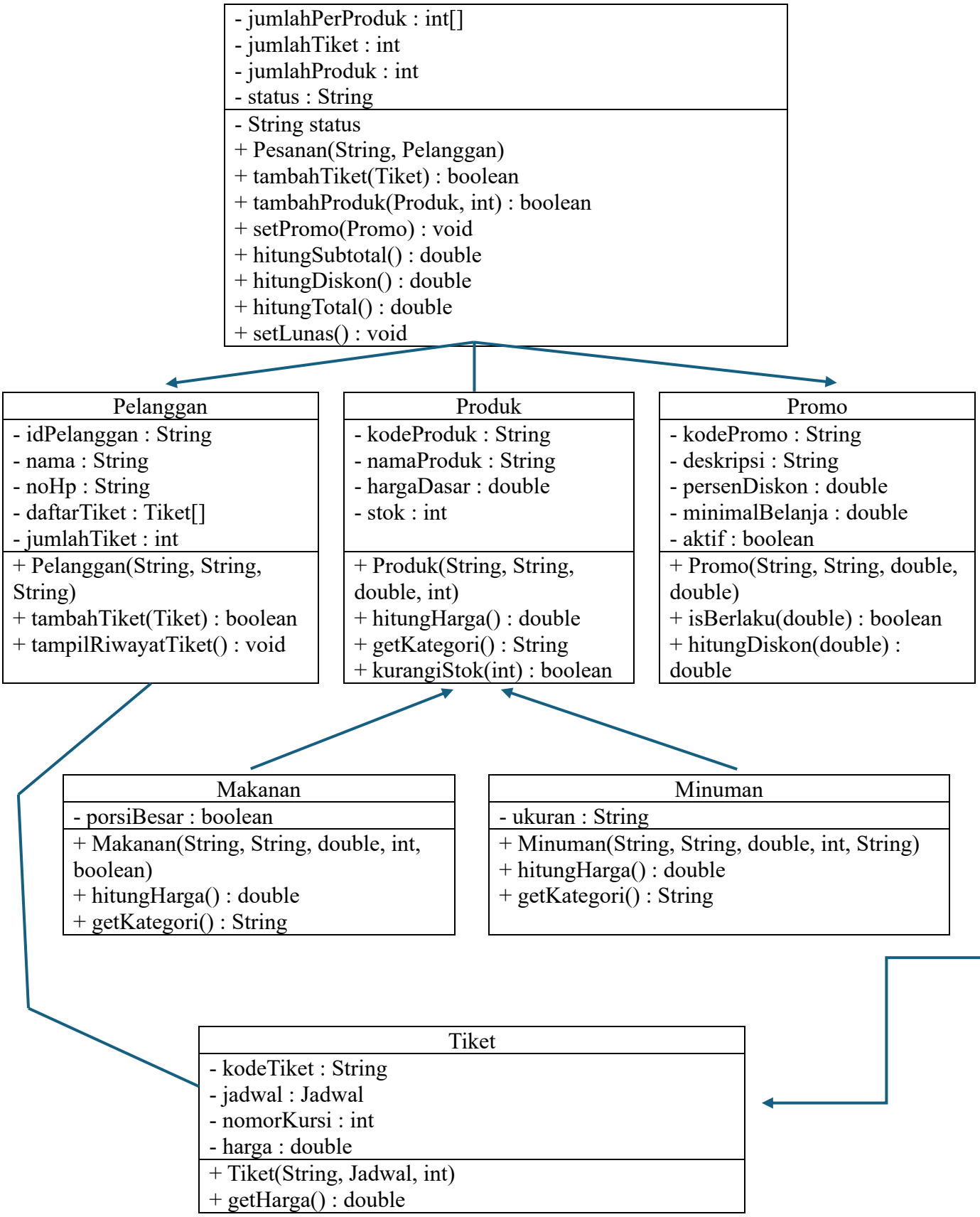
9. Program harus dapat dikompilasi dan dijalankan.

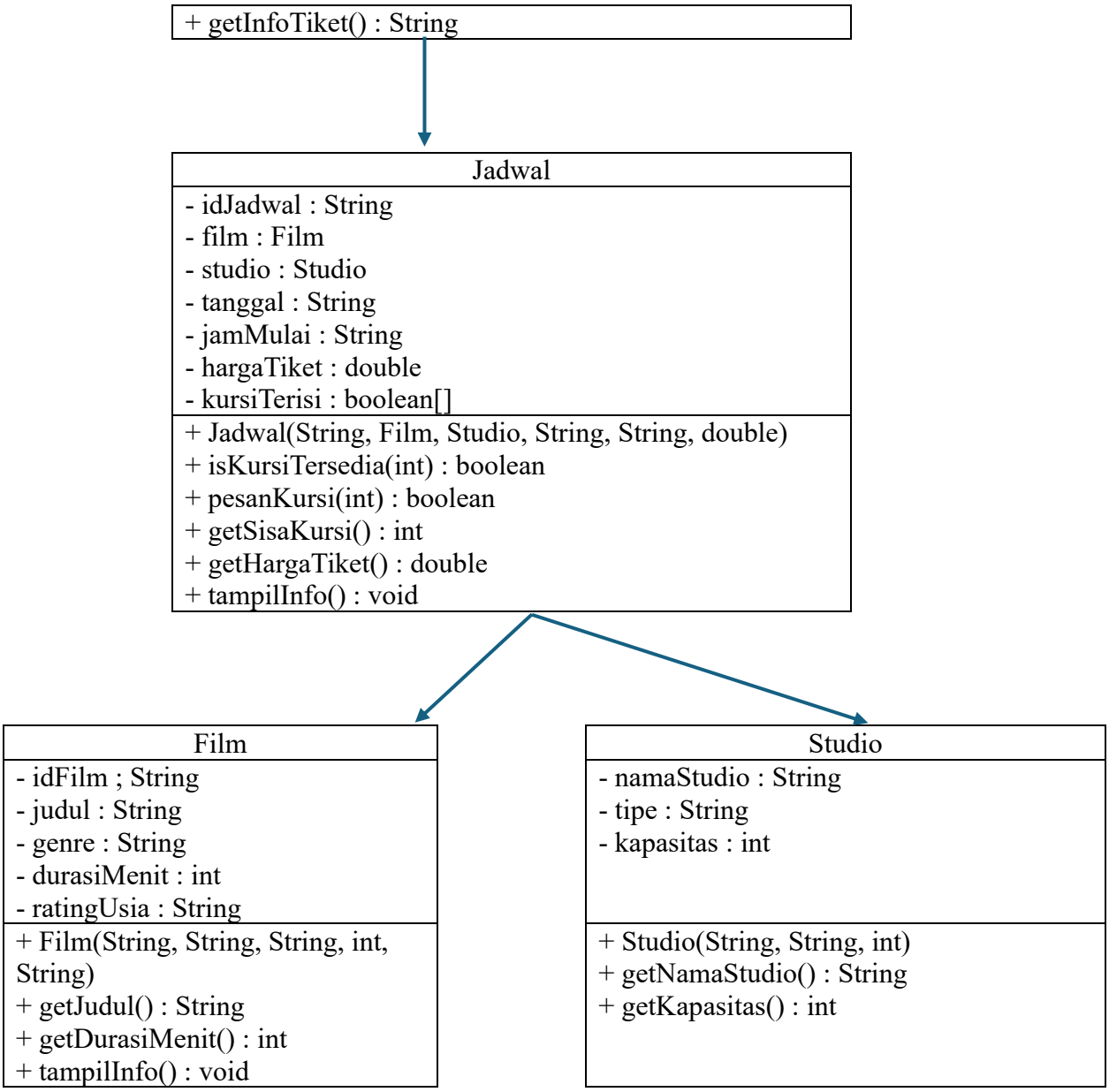
10. ntuk pembagian sutdi kasus, silahkan modulus 2 nomor terkahir NRP Anda dengan 6 (mod6), jika sisa bagi NRP Anda adalah 0 maka studikasus yang harus Anda kerjakan adalah studi kasus 6.

- 47 % 6 = 5 berarti Bioskop 212

UML DIAGRAM


Kasir
- idKasir : String
- namaKasir : String
- counterPesanan : int
- counterTiket : int
+ Kasir(String, String)
+ buatPesanan(Pelanggan) : Pesanan
+ jualTiket(Pesanan, Jadwal, int) : boolean
+ tambahProdukKePesanan(Pesanan, Produk, int) : boolean
+ terapkanPromo(Pesanan, Promo) : boolean
+ prosesPembayaran(Pembayaran) : boolean
+ cetakStruk(Pesanan) : void
Pembayaran
- idPembayaran : String
- pesanan : Pesanan
- lunas : Boolean
+ Pembayaran(String, Pesanan)
+ hitungBiayaAdmin() : double
+ getMetode() : String
+ proses() : boolean
+ getTotalBayar() : double
PembayaranTunai
- uangDiterima : double
+ PembayaranTunai(String, Pesanan, double)
+ hitungBiayaAdmin() : double
+ getMetode() : String
+ proses() : boolean
+ getKembalian() : double
PembayaranQRIS
- penyediaQR : String
+ PembayaranQRIS(String, Pesanan, String)
+ hitungBiayaAdmin() : double
+ getMetode() : String
+ proses() : boolean
Pesanan
- idPesanan : String
- pelanggan : Pelanggan
- promo : Promo
- daftarTiket : Tiket[]
- daftarProduk : Produk[]


- jumlahPerProduk : int[]
- jumlahTiket : int
- jumlahProduk : int
- status : String
- String status
+ Pesanan(String, Pelanggan)
+ tambahTiket(Tiket) : boolean
+ tambahProduk(Produk, int) : boolean
+ setPromo(Promo) : void
+ hitungSubtotal() : double
+ hitungDiskon() : double
+ hitungTotal() : double
+ setLunas() : void
Pelanggan
- idPelanggan : String
- nama : String
- noHp : String
- daftarTiket : Tiket[]
- jumlahTiket : int
+ Pelanggan(String, String, String)
+ tambahTiket(Tiket) : boolean
+ tampilRiwayatTiket() : void
Produk
- kodeProduk : String
- namaProduk : String
- hargaDasar : double
- stok : int
+ Produk(String, String, double, int)
+ hitungHarga() : double
+ getKategori() : String
+ kurangiStok(int) : boolean
Promo
- kodePromo : String
- deskripsi : String
- persenDiskon : double
- minimalBelanja : double
- aktif : boolean
+ Promo(String, String, double, double)
+ isBerlaku(double) : boolean
+ hitungDiskon(double) : double
Makanan
- porsiBesar : boolean
+ Makanan(String, String, double, int, boolean)
+ hitungHarga() : double
+ getKategori() : String
Minuman
- ukuran : String
+ Minuman(String, String, double, int, String)
+ hitungHarga() : double
+ getKategori() : String
Tiket
- kodeTiket : String
- jadwal : Jadwal
- nomorKursi : int
- harga : double
+ Tiket(String, Jadwal, int)
+ getHarga() : double


+ getInfoTiket() : String
Jadwal
- idJadwal : String
- film : Film
- studio : Studio
- tanggal : String
- jamMulai : String
- hargaTiket : double
- kursiTerisi : boolean[]
+ Jadwal(String, Film, Studio, String, String, double)
+ isKursiTersedia(int) : boolean
+ pesanKursi(int) : boolean
+ getSisaKursi() : int
+ getHargaTiket() : double
+ tampilInfo() : void
Film
- idFilm ; String
- judul : String
- genre : String
- durasiMenit : int
- ratingUsia : String
+ Film(String, String, String, int, String)
+ getJudul() : String
+ getDurasiMenit() : int
+ tampilInfo() : void
Studio
- namaStudio : String
- tipe : String
- kapasitas : int
+ Studio(String, String, int)
+ getNamaStudio() : String
+ getKapasitas() : int

Struktur Folder

UTS STUCY CASE/

```
├── laporan/
└── src/
    ├── Main.java
    ├── Film.java
    ├── Studio.java
    ├── Jadwal.java
    ├── Tiket.java
    ├── Pelanggan.java
    ├── Produk.java
    ├── Makanan.java
    ├── Minuman.java
    ├── Promo.java
    ├── Pesanan.java
    ├── Kasir.java
    ├── Pembayaran.java
    ├── PembayaranTunai.java
    └── PembayaranQRIS.java
```

Kode

Main.java

```
public class Main {
    public static void main(String[] args) {
        Film[] films = {
            new Film("F01", "Laskar Pelangi 2", "Drama", 120, "SU"),
            new Film("F02", "Galaksi Terakhir", "Sci-Fi", 135, "13+"),
            new Film("F03", "Malam Jumat Kliwon", "Horor", 100, "17+")
        };
        Studio[] studios = {
            new Studio("Studio 1", "Reguler", 30),
            new Studio("Studio 2", "Premiere", 20)
        };
        Jadwal[] jadwals = {
            new Jadwal("J01", films[0], studios[0], "2026-10-15", "13:00",
50000),
            new Jadwal("J02", films[1], studios[1], "2026-10-15", "16:00",
75000),
            new Jadwal("J03", films[2], studios[0], "2026-10-15", "19:00",
50000)
        };
        Produk[] produks = {
            new Makanan("M01", "Popcorn Asin", 30000, 20, false),
            new Makanan("M02", "Popcorn Karamel", 35000, 20, true),
            new Minuman("D01", "Cola", 15000, 20, "L"),
            new Minuman("D02", "Air Mineral", 8000, 30, "S")
        };
        Promo[] promos = {
            new Promo("HEMAT10", "Diskon 10% min. belanja Rp100000", 10,
100000),
            new Promo("BIGDEAL", "Diskon 20% min. belanja Rp300000", 20,
300000)
        };
        Kasir kasir = new Kasir("K01", "Rina");
        Pelanggan budi = new Pelanggan("P01", "Budi", "0811111111");

        System.out.println("##### JADWAL FILM #####");
        for (int i = 0; i < jadwals.length; i++) {
            jadwals[i].tampilInfo();
        }
        System.out.println("\n##### MENU #####");
        for (int i = 0; i < produks.length; i++) {
            System.out.println(produks[i].getKategori() + " | " +
produks[i].getNamaProduk()
```

```
                    + " | Rp" + Math.round(produks[i].hitungHarga())
                    + " | stok " + produks[i].getStok());
        }

        System.out.println("\n##### SIMULASI: Budi beli 2 tiket + makanan,
bayar #####");
        Pesanan p1 = kasir.buatPesanan(budi);
        System.out.println("Tiket kursi 5 : " + (kasir.jualTiket(p1,
jadwals[0], 5) ? "berhasil" : "gagal"));
        System.out.println("Tiket kursi 6 : " + (kasir.jualTiket(p1,
jadwals[0], 6) ? "berhasil" : "gagal"));
        System.out.println("Popcorn Karamel x1: " +
(kasir.tambahProdukKePesanan(p1, produks[1], 1) ? "berhasil" : "gagal"));
        System.out.println("Cola L x1 : " + (kasir.tambahProdukKePesanan(p1,
produks[2], 1) ? "berhasil" : "gagal"));
        System.out.println("Cola x99 (stok tidak cukup): " +
(kasir.tambahProdukKePesanan(p1, produks[2], 99) ? "berhasil" : "gagal"));
        System.out.println("Promo HEMAT10 : " + (kasir.terapkanPromo(p1,
promos[0]) ? "diterapkan" : "tidak memenuhi syarat"));

        Pembayaran bayar1 = new PembayaranTunai("PAY-001", p1, 150000);
        if (kasir.prosesPembayaran(bayar1)) {
            System.out.println("Pembayaran " + bayar1.getMetode() + " berhasil.
Kembalian: Rp"
                    + Math.round(((PembayaranTunai) bayar1).getKembalian()));
        }
        kasir.cetakStruk(p1);

    }
}
```

Film.java

```
public class Film {
    private String idFilm;
    private String judul;
    private String genre;
    private int durasiMenit;
    private String ratingUsia;

    public Film(String idFilm, String judul, String genre, int durasiMenit,
String ratingUsia) {
        this.idFilm = idFilm;
        this.judul = judul;
        this.genre = genre;
```

```
        setDurasiMenit(durasiMenit);
        this.ratingUsia = ratingUsia;
    }

    public String getIdFilm() { return idFilm; }
    public String getJudul() { return judul; }
    public String getGenre() { return genre; }
    public int getDurasiMenit() { return durasiMenit; }
    public String getRatingUsia() { return ratingUsia; }

    public void setDurasiMenit(int durasiMenit) {
        if (durasiMenit <= 0) {
            throw new IllegalArgumentException("Durasi film harus lebih dari 0
menit");
        }
        this.durasiMenit = durasiMenit;
    }

    public void tampilInfo() {
        System.out.println(judul + " (" + genre + ", " + durasiMenit + " menit,
" + ratingUsia + ")");
    }
}
```

Jadwal.java

```
public class Jadwal {
    private String idJadwal;
    private Film film;
    private Studio studio;
    private String tanggal;
    private String jamMulai;
    private double hargaTiket;
    private boolean[] kursiTerisi; // index 0 = kursi nomor 1

    public Jadwal(String idJadwal, Film film, Studio studio, String tanggal,
String jamMulai, double hargaTiket) {
        this.idJadwal = idJadwal;
        this.film = film;
        this.studio = studio;
        this.tanggal = tanggal;
        this.jamMulai = jamMulai;
        if (hargaTiket < 0) {
            throw new IllegalArgumentException("Harga tiket tidak boleh
negatif");
```

```
        }
        this.hargaTiket = hargaTiket;
        this.kursiTerisi = new boolean[studio.getKapasitas()];
    }

    public String getIdJadwal() { return idJadwal; }
    public Film getFilm() { return film; }
    public Studio getStudio() { return studio; }
    public String getTanggal() { return tanggal; }
    public String getJamMulai() { return jamMulai; }
    public double getHargaTiket() { return hargaTiket; }

    public boolean isKursiTersedia(int nomorKursi) {
        if (nomorKursi < 1 || nomorKursi > kursiTerisi.length) {
            return false;
        }
        return !kursiTerisi[nomorKursi - 1];
    }

    public boolean pesanKursi(int nomorKursi) {
        if (!isKursiTersedia(nomorKursi)) {
            return false;
        }
        kursiTerisi[nomorKursi - 1] = true;
        return true;
    }

    public int getSisaKursi() {
        int sisa = 0;
        for (int i = 0; i < kursiTerisi.length; i++) {
            if (!kursiTerisi[i]) {
                sisa++;
            }
        }
        return sisa;
    }

    public void tampilInfo() {
        System.out.println("[" + idJadwal + "] " + film.getJudul()
                + " | " + studio.getNamaStudio() + " (" + studio.getTipe() +
")"
                + " | " + tanggal + " " + jamMulai
                + " | Rp" + Math.round(hargaTiket)
                + " | Sisa kursi: " + getSisaKursi() + "/" +
studio.getKapasitas());
```

```
    }
}
```

Kasir.java

```
public class Kasir {
    private String idKasir;
    private String namaKasir;
    private int counterPesanan;
    private int counterTiket;

    public Kasir(String idKasir, String namaKasir) {
        this.idKasir = idKasir;
        this.namaKasir = namaKasir;
        this.counterPesanan = 0;
        this.counterTiket = 0;
    }

    public String getIdKasir() { return idKasir; }
    public String getNamaKasir() { return namaKasir; }

    public Pesanan buatPesanan(Pelanggan pelanggan) {
        counterPesanan++;
        return new Pesanan(String.format("PSN-%03d", counterPesanan),
pelanggan);
    }

    public boolean jualTiket(Pesanan pesanan, Jadwal jadwal, int nomorKursi) {
        if (pesanan.isLunas() || pesanan.isTiketPenuh() ||
pesanan.getPelanggan().isRiwayatPenuh()) {
            return false;
        }
        if (!jadwal.pesanKursi(nomorKursi)) { // false jika kursi terisi /
tidak valid
            return false;
        }
        counterTiket++;
        Tiket tiket = new Tiket(String.format("TKT-%03d", counterTiket),
jadwal, nomorKursi);
        pesanan.tambahTiket(tiket);
        pesanan.getPelanggan().tambahTiket(tiket);
        return true;
    }
```

```java
    public boolean tambahProdukKePesanan(Pesanan pesanan, Produk produk, int
jumlah) {
        if (!pesanan.tambahProduk(produk, jumlah)) {
            return false;
        }
        return produk.kurangiStok(jumlah);
    }

    public boolean terapkanPromo(Pesanan pesanan, Promo promo) {
        if (promo == null || !promo.isBerlaku(pesanan.hitungSubtotal())) {
            return false;
        }
        pesanan.setPromo(promo);
        return true;
    }

    public boolean prosesPembayaran(Pembayaran pembayaran) {
        Pesanan pesanan = pembayaran.getPesanan();
        if (pesanan.isLunas()) {
            return false;
        }
        boolean berhasil = pembayaran.proses(); // polymorphism
        if (berhasil) {
            pesanan.setLunas();
        }
        return berhasil;
    }

    public void cetakStruk(Pesanan pesanan) {
        System.out.println("========================================");
        System.out.println("              BIOSKOP 212");
        System.out.println("========================================");
        System.out.println("Pesanan  : " + pesanan.getIdPesanan());
        System.out.println("Kasir    : " + namaKasir);
        System.out.println("Pelanggan: " + pesanan.getPelanggan().getNama());
        System.out.println("----------------------------------------");
        for (int i = 0; i < pesanan.getJumlahTiket(); i++) {
            System.out.println("Tiket " + pesanan.getTiket(i).getInfoTiket());
        }
        for (int i = 0; i < pesanan.getJumlahJenisProduk(); i++) {
            Produk p = pesanan.getProduk(i);
            int qty = pesanan.getQtyProduk(i);
            System.out.println(p.getKategori() + " " + p.getNamaProduk() + " x"
+ qty
                    + " = Rp" + Math.round(p.hitungHarga() * qty));
```

```
        }
        System.out.println("----------------------------------------");
        System.out.println("Subtotal : Rp" +
Math.round(pesanan.hitungSubtotal()));
        if (pesanan.getPromo() != null) {
            System.out.println("Diskon (" + pesanan.getPromo().getKodePromo() +
"): -Rp"
                    + Math.round(pesanan.hitungDiskon()));
        }
        System.out.println("TOTAL    : Rp" +
Math.round(pesanan.hitungTotal()));
        System.out.println("Status   : " + pesanan.getStatus());
        System.out.println("========================================");
    }
}
```

Makanan.java

```
public class Makanan extends Produk {
    private boolean porsiBesar;

    public Makanan(String kodeProduk, String namaProduk, double hargaDasar, int
stok, boolean porsiBesar) {
        super(kodeProduk, namaProduk, hargaDasar, stok);
        this.porsiBesar = porsiBesar;
    }

    public boolean isPorsiBesar() { return porsiBesar; }

    @Override
    public double hitungHarga() {
        if (porsiBesar) {
            return getHargaDasar() * 1.2; // asumsi: porsi besar +20%
        }
        return getHargaDasar();
    }

    @Override
    public String getKategori() {
        return "Makanan";
    }
}
```

Minuman.java

```java
public class Minuman extends Produk {
    private String ukuran; // "S", "M", "L"

    public Minuman(String kodeProduk, String namaProduk, double hargaDasar, int
stok, String ukuran) {
        super(kodeProduk, namaProduk, hargaDasar, stok);
        setUkuran(ukuran);
    }

    public String getUkuran() { return ukuran; }

    public void setUkuran(String ukuran) {
        String u = ukuran.toUpperCase();
        if (!u.equals("S") && !u.equals("M") && !u.equals("L")) {
            throw new IllegalArgumentException("Ukuran harus S, M, atau L");
        }
        this.ukuran = u;
    }

    @Override
    public double hitungHarga() {
        if (ukuran.equals("M")) {
            return getHargaDasar() + 3000; // asumsi
        } else if (ukuran.equals("L")) {
            return getHargaDasar() + 6000; // asumsi
        }
        return getHargaDasar();
    }

    @Override
    public String getKategori() {
        return "Minuman";
    }
}
```

Pelanggan.java

```java
public class Pelanggan {
    private static final int MAKS_TIKET = 10;

    private String idPelanggan;
    private String nama;
    private String noHp;
    private Tiket[] daftarTiket;
    private int jumlahTiket;
```

```
    public Pelanggan(String idPelanggan, String nama, String noHp) {
        this.idPelanggan = idPelanggan;
        this.nama = nama;
        this.noHp = noHp;
        this.daftarTiket = new Tiket[MAKS_TIKET];
        this.jumlahTiket = 0;
    }

    public String getIdPelanggan() { return idPelanggan; }
    public String getNama() { return nama; }
    public String getNoHp() { return noHp; }
    public int getJumlahTiket() { return jumlahTiket; }

    public boolean isRiwayatPenuh() {
        return jumlahTiket >= daftarTiket.length;
    }

    public boolean tambahTiket(Tiket tiket) {
        if (tiket == null || isRiwayatPenuh()) {
            return false;
        }
        daftarTiket[jumlahTiket] = tiket;
        jumlahTiket++;
        return true;
    }

    public void tampilRiwayatTiket() {
        System.out.println("Riwayat tiket " + nama + " (" + jumlahTiket + "
tiket):");
        if (jumlahTiket == 0) {
            System.out.println("  (belum ada tiket)");
        }
        for (int i = 0; i < jumlahTiket; i++) {
            System.out.println("  " + (i + 1) + ". " +
daftarTiket[i].getInfoTiket());
        }
    }
}
```

Pembayaran.java

```
public abstract class Pembayaran {
    private String idPembayaran;
    private Pesanan pesanan;
```

```
    private boolean lunas;

    public Pembayaran(String idPembayaran, Pesanan pesanan) {
        this.idPembayaran = idPembayaran;
        this.pesanan = pesanan;
        this.lunas = false;
    }

    public String getIdPembayaran() { return idPembayaran; }
    public Pesanan getPesanan() { return pesanan; }
    public boolean isLunas() { return lunas; }

    protected void setLunas(boolean lunas) {
        this.lunas = lunas;
    }

    public double getTotalBayar() {
        return pesanan.hitungTotal() + hitungBiayaAdmin();
    }

    public abstract double hitungBiayaAdmin();

    public abstract String getMetode();

    public abstract boolean proses();
}
```

PembayaranQRIS.java

```
public class PembayaranQRIS extends Pembayaran {
    private String penyediaQR;

    public PembayaranQRIS(String idPembayaran, Pesanan pesanan, String
penyediaQR) {
        super(idPembayaran, pesanan);
        this.penyediaQR = penyediaQR;
    }

    public String getPenyediaQR() { return penyediaQR; }

    @Override
    public double hitungBiayaAdmin() {
        return getPesanan().hitungTotal() * 0.007; // asumsi 0,7%
    }
```

```
    @Override
    public String getMetode() {
        return "QRIS (" + penyediaQR + ")";
    }

    @Override
    public boolean proses() {
        // Simulasi: selalu berhasil
        setLunas(true);
        return true;
    }
}
```

PembayaranTunai.java

```
public class PembayaranTunai extends Pembayaran {
    private double uangDiterima;

    public PembayaranTunai(String idPembayaran, Pesanan pesanan, double
uangDiterima) {
        super(idPembayaran, pesanan);
        this.uangDiterima = uangDiterima;
    }

    public double getUangDiterima() { return uangDiterima; }

    @Override
    public double hitungBiayaAdmin() {
        return 0;
    }

    @Override
    public String getMetode() {
        return "Tunai";
    }

    @Override
    public boolean proses() {
        if (uangDiterima >= getTotalBayar()) {
            setLunas(true);
            return true;
        }
        return false;
    }
```

```
    public double getKembalian() {
        if (!isLunas()) {
            return 0;
        }
        return uangDiterima - getTotalBayar();
    }
}
```

Pesanan.java

```
public class Pesanan {
    private static final int MAKS_TIKET = 5;
    private static final int MAKS_PRODUK = 10;

    private String idPesanan;
    private Pelanggan pelanggan;
    private Promo promo;
    private Tiket[] daftarTiket;
    private Produk[] daftarProduk;
    private int[] jumlahPerProduk;
    private int jumlahTiket;
    private int jumlahProduk;
    private String status;

    public Pesanan(String idPesanan, Pelanggan pelanggan) {
        this.idPesanan = idPesanan;
        this.pelanggan = pelanggan;
        this.promo = null;
        this.daftarTiket = new Tiket[MAKS_TIKET];
        this.daftarProduk = new Produk[MAKS_PRODUK];
        this.jumlahPerProduk = new int[MAKS_PRODUK];
        this.jumlahTiket = 0;
        this.jumlahProduk = 0;
        this.status = "BELUM_BAYAR";
    }

    public String getIdPesanan() { return idPesanan; }
    public Pelanggan getPelanggan() { return pelanggan; }
    public Promo getPromo() { return promo; }
    public String getStatus() { return status; }
    public boolean isLunas() { return status.equals("LUNAS"); }
    public int getJumlahTiket() { return jumlahTiket; }
    public int getJumlahJenisProduk() { return jumlahProduk; }
    public Tiket getTiket(int index) { return daftarTiket[index]; }
    public Produk getProduk(int index) { return daftarProduk[index]; }
```

```java
    public int getQtyProduk(int index) { return jumlahPerProduk[index]; }

    public boolean isTiketPenuh() {
        return jumlahTiket >= daftarTiket.length;
    }

    public boolean tambahTiket(Tiket tiket) {
        if (tiket == null || isLunas() || isTiketPenuh()) {
            return false;
        }
        daftarTiket[jumlahTiket] = tiket;
        jumlahTiket++;
        return true;
    }

    // Stok hanya dicek di sini. Pengurangan stok dilakukan Kasir setelah
berhasil.
    public boolean tambahProduk(Produk produk, int jumlah) {
        if (produk == null || jumlah <= 0 || isLunas() || jumlahProduk >=
daftarProduk.length) {
            return false;
        }
        if (produk.getStok() < jumlah) {
            return false;
        }
        daftarProduk[jumlahProduk] = produk;
        jumlahPerProduk[jumlahProduk] = jumlah;
        jumlahProduk++;
        return true;
    }

    public void setPromo(Promo promo) {
        if (!isLunas()) {
            this.promo = promo;
        }
    }

    public double hitungSubtotal() {
        double subtotal = 0;
        for (int i = 0; i < jumlahTiket; i++) {
            subtotal += daftarTiket[i].getHarga();
        }
        for (int i = 0; i < jumlahProduk; i++) {
            subtotal += daftarProduk[i].hitungHarga() * jumlahPerProduk[i]; //
polymorphism
```

```
        }
        return subtotal;
    }

    public double hitungDiskon() {
        if (promo == null) {
            return 0;
        }
        return promo.hitungDiskon(hitungSubtotal());
    }

    public double hitungTotal() {
        return hitungSubtotal() - hitungDiskon();
    }

    public void setLunas() {
        this.status = "LUNAS";
    }
}
```

Produk.java

```
public abstract class Produk {
    private String kodeProduk;
    private String namaProduk;
    private double hargaDasar;
    private int stok;

    public Produk(String kodeProduk, String namaProduk, double hargaDasar, int
stok) {
        this.kodeProduk = kodeProduk;
        this.namaProduk = namaProduk;
        if (hargaDasar < 0 || stok < 0) {
            throw new IllegalArgumentException("Harga dan stok tidak boleh
negatif");
        }
        this.hargaDasar = hargaDasar;
        this.stok = stok;
    }

    public String getKodeProduk() { return kodeProduk; }
    public String getNamaProduk() { return namaProduk; }
    public double getHargaDasar() { return hargaDasar; }
    public int getStok() { return stok; }
```

```
    public boolean kurangiStok(int jumlah) {
        if (jumlah <= 0 || jumlah > stok) {
            return false;
        }
        stok -= jumlah;
        return true;
    }

    public abstract double hitungHarga();

    public abstract String getKategori();
}
```

Promo.java

```
public class Promo {
    private String kodePromo;
    private String deskripsi;
    private double persenDiskon;
    private double minimalBelanja;
    private boolean aktif;

    public Promo(String kodePromo, String deskripsi, double persenDiskon,
double minimalBelanja) {
        this.kodePromo = kodePromo;
        this.deskripsi = deskripsi;
        if (persenDiskon < 0 || persenDiskon > 100) {
            throw new IllegalArgumentException("Persen diskon harus 0 - 100");
        }
        this.persenDiskon = persenDiskon;
        this.minimalBelanja = minimalBelanja;
        this.aktif = true;
    }

    public String getKodePromo() { return kodePromo; }
    public String getDeskripsi() { return deskripsi; }
    public double getPersenDiskon() { return persenDiskon; }
    public boolean isAktif() { return aktif; }
    public void setAktif(boolean aktif) { this.aktif = aktif; }

    public boolean isBerlaku(double subtotal) {
        return aktif && subtotal >= minimalBelanja;
    }

    public double hitungDiskon(double subtotal) {
```

```
        if (!isBerlaku(subtotal)) {
            return 0;
        }
        return subtotal * persenDiskon / 100;
    }
}
```

Studio.java

```
public class Studio {
    private String namaStudio;
    private String tipe;
    private int kapasitas;

    public Studio(String namaStudio, String tipe, int kapasitas) {
        this.namaStudio = namaStudio;
        this.tipe = tipe;
        if (kapasitas <= 0) {
            throw new IllegalArgumentException("Kapasitas studio harus lebih
dari 0");
        }
        this.kapasitas = kapasitas;
    }

    public String getNamaStudio() { return namaStudio; }
    public String getTipe() { return tipe; }
    public int getKapasitas() { return kapasitas; }
}
```

Tiket.java

```
public class Tiket {
    private String kodeTiket;
    private Jadwal jadwal;
    private int nomorKursi;
    private double harga;

    public Tiket(String kodeTiket, Jadwal jadwal, int nomorKursi) {
        this.kodeTiket = kodeTiket;
        this.jadwal = jadwal;
        this.nomorKursi = nomorKursi;
        this.harga = jadwal.getHargaTiket(); // harga dikunci saat tiket dibuat
    }

    public String getKodeTiket() { return kodeTiket; }
```

```
    public Jadwal getJadwal() { return jadwal; }
    public int getNomorKursi() { return nomorKursi; }
    public double getHarga() { return harga; }

    public String getInfoTiket() {
        return kodeTiket + " | " + jadwal.getFilm().getJudul()
                + " | " + jadwal.getStudio().getNamaStudio()
                + " | " + jadwal.getTanggal() + " " + jadwal.getJamMulai()
                + " | Kursi " + nomorKursi
                + " | Rp" + Math.round(harga);
    }
}
```

Output

```
##### JADWAL FILM #####
[J01] Laskar Pelangi 2 | Studio 1 (Reguler) | 2026-10-15 13:00 | Rp50000 | Sisa kursi: 30/30
[J02] Galaksi Terakhir | Studio 2 (Premiere) | 2026-10-15 16:00 | Rp75000 | Sisa kursi: 20/20
[J03] Malam Jumat Kliwon | Studio 1 (Reguler) | 2026-10-15 19:00 | Rp50000 | Sisa kursi: 30/30

##### MENU #####
Makanan | Popcorn Asin | Rp30000 | stok 20
Makanan | Popcorn Karamel | Rp42000 | stok 20
Minuman | Cola | Rp21000 | stok 20
Minuman | Air Mineral | Rp8000 | stok 30

##### SIMULASI: Budi beli 2 tiket + makanan, bayar #####
Tiket kursi 5 : berhasil
Tiket kursi 6 : berhasil
Popcorn Karamel x1: berhasil
Cola L x1 : berhasil
Cola x99 (stok tidak cukup): gagal
Promo HEMAT10 : diterapkan
Pembayaran Tunai berhasil. Kembalian: Rp3300
========================================
             BIOSKOP 212
========================================
Pesanan  : PSN-001
Kasir    : Rina
Pelanggan: Budi
----------------------------------------
Tiket TKT-001 | Laskar Pelangi 2 | Studio 1 | 2026-10-15 13:00 | Kursi 5 | Rp50000
Tiket TKT-002 | Laskar Pelangi 2 | Studio 1 | 2026-10-15 13:00 | Kursi 6 | Rp50000
Makanan Popcorn Karamel x1 = Rp42000
Minuman Cola x1 = Rp21000
----------------------------------------
Subtotal : Rp163000
Diskon (HEMAT10): -Rp16300
TOTAL    : Rp146700
Status   : LUNAS
========================================
```

Kesimpulan

Program sudah menerapkan prinsip OOP pada studi kasus Bioskop 212 melalui 15 class Java yang saling berinteraksi. Enkapsulasi diterapkan dengan atribut private yang diakses lewat getter dan dilengkapi validasi. Polymorphism diterapkan lewat class abstract Produk (subclass Makanan dan Minuman) serta Pembayaran (subclass PembayaranTunai dan PembayaranQRIS). Array of object diterapkan untuk menyimpan dan mengolah banyak data, seperti Film[], Jadwal[], Produk[], dan Tiket[]. Program juga sudah menjalankan alur bisnis utama, yaitu pembuatan pesanan, penjualan tiket, penerapan promo, pembayaran, dan pencetakan struk, serta berhasil dikompilasi dan dijalankan melalui main().